Homework 4
Computer System Organization
ECE 361
Fall 2026

Overview

Homework 4 is your first abstract data type: a **tally** that counts how often each name occurs, such as the events your device sees. Callers use it only through its header; only `tally.c` knows what is inside. It is due on **Sun, Oct 25, 11:59pm**, pushed to `hw04/`. AI tools are allowed. Your Shine about your code is due by **Wed, Oct 28, 11:59pm**, without any supporting materials. Satisfactory / Not Yet, as before.

Part 1: The tally ADT

In `hw04/tally.h`, declare exactly this interface, with a comment per function on what it takes, returns, and assumes:

```
#define TALLY_MAX     4    /* tallies that can exist at the same time */
#define TALLY_NAMES  16    /* different names one tally can hold */
#define TALLY_LEN    15    /* longest name, without the '\0' */

typedef struct tally tally_t;      /* opaque */

tally_t *tally_create(void);
void     tally_destroy(tally_t *t);
int      tally_add(tally_t *t, const char *name);
int      tally_count(const tally_t *t, const char *name);
int      tally_size(const tally_t *t);
void     tally_print(const tally_t *t, FILE *out);
```

Table 1. What each function does.

Function	What it does
`tally_create`	a new, empty tally; `NULL` if `TALLY_MAX` tallies are already in use
`tally_destroy`	gives the tally back so that `tally_create` can hand it out again; `NULL` does nothing
`tally_add`	counts one more occurrence of `name` and returns its new count; returns −1 and changes nothing if `t` or `name` is `NULL`, `name` is empty or longer than `TALLY_LEN`, or `name` is new and the tally is full
`tally_count`	how often `name` was added: 0 if never, or for `NULL`
`tally_size`	how many different names the tally holds: 0 for `NULL`
`tally_print`	one line per name, `name count`, in alphabetical (`strcmp`) order

In `hw04/tally.c`:

- **Complete the type here and nowhere else:** `struct tally` holds an array of entries, each a name and a count, **kept sorted by name**. A new name goes where it belongs; the later entries move up by one.
- **No malloc (that is week 7):** the tallies live in a `static` array of `TALLY_MAX` inside `tally.c`, each marked in use or free.
- **The tally copies each name**, safely: check the length first, and never write past an array. No `strcpy` or `strncpy` without a length check.

Part 2: A program, tests, and a Makefile

- `hw04/events.c`: reads standard input line by line with `fgets`, removes the newline, skips blank lines and lines that start with `#`, adds every other line to one tally, then prints the tally and one last

line, `names: N, lines skipped: S`, where S counts the lines `tally_add` refused. Example: `./events < data/door.txt`, with a `data/door.txt` of your own.

- `hw04/tests/test_tally.c` tests every function, at least: counting up; a name never added; an empty name; a name of exactly `TALLY_LEN` characters and one of `TALLY_LEN + 1`; `NULL` arguments; a full tally (a new name is refused, an existing one still counts); `TALLY_MAX` tallies, then one more (`NULL`); create after destroy; two tallies are independent; the print order. PASS or FAIL per test, a summary, and a nonzero exit status if any test fails.
- `hw04/Makefile`: `make` builds `events` with `gcc -std=c11 -Wall -Wextra` and no warnings, `make test` builds and runs the tests, `make clean` removes what was built. Each `.o` lists the headers it includes.

Part 3: What the opaque type prevents

In `hw04/README.md`, after how to build and test: try `t->n` (or any member of yours) in a file that includes only `tally.h`, copy the compiler's error message into the README, and explain in one paragraph what the opaque type prevents and why that is useful. Then remove the line: the code you push must compile.

Your project: a next step (not graded)

Scaffolding for your device, not part of HW4: it is not graded, and the HW4 Shine does not ask about it. After Thursday's studio, write your `reading_t` in `project/sensor.h` and your `record_t` and opaque `store_t` in `project/store.h`, as your project sheet describes, and implement `store.c` as a sorted array for now, the way the tally keeps its names. Your tree replaces the inside in week 8.

Deliverable

Push by Sun, Oct 25, 11:59pm: `hw04/`: `tally.h`, `tally.c`, `events.c`, `data/`, `tests/test_tally.c`, `Makefile`, `README.md` (how to build and test, and Part 3), `AI_USAGE.md` (tools, for what, one thing a model got wrong).

Shine

The window opens Mon, Oct 26, in the morning; due Wed, Oct 28, 11:59pm; about 10 to 15 minutes, about your own code. Expect questions like: "Walk me through your `tally_add` when a new name goes in the middle: what moves, and why in that order?" and "Why can `events.c` not look inside a `tally_t`, and what would break if it could?" No notes, no second screen, no AI, no other person; webcam on.

Requirements for Satisfactory

- Every deliverable is in your repository at the deadline.
- `make` builds with no errors and no warnings, `make test` runs and all tests pass, `make clean` works.
- `tally_t` is opaque: only `tally.c` completes `struct tally`; there is no `malloc` and no global outside the pool; names are kept sorted and copied safely.
- Your Shine is completed by the deadline and meets the bar.